Overview

In addition to writing skills with Misty's JavaScript and .NET SDKs, you can also write powerful, remote-running robot applications using Misty's HTTP API and WebSocket server.

Writing a robot application with Misty's HTTP API typically involves two things:

- sending requests to Misty's HTTP API endpoints
- getting live data from Misty's sensors and other event types via WebSocket connections

The topics below provide the information you need to perform those tasks in your robot applications. For information about each individual API request, see the HTTP API reference documentation. For information about the individual event types you can subscribe to, see the Event Types documentation.

Sending Requests to Misty

The topics in this section explain how to use Misty's HTTP API to control Misty II, change her settings, and manage her data. This documentation assumes you're already familiar with the basic concepts of using a HTTP API. If you've never used a HTTP API, we recommend learning the fundamentals before you continue.

What You Need

To use Misty's HTTP API, you need:

- a Misty II robot, powered on and connected to your local area network
- a way to send HTTP requests - for example, by writing HTTP client code, or by using the API Explorer or a REST client tool on a laptop connected to the same network as Misty

> **Note:** Misty's HTTP API does not use keys or credentials to authorize requests. Any device that can discover Misty's IP address and send HTTP requests can send Misty commands and stream data from Misty's WebSocket server. Keep this in mind when using Misty on your local network or exposing Misty's IP address to the world wide web.

Base URL

The base URL for all requests in this API is `<robot-ip-address>/api/`. For example:

```
http://<robot-ip-address>/api/<Endpoint>
```

Markup

Replace `<robot-ip-address>` with one of the following:

- Misty's Wi-Fi IP address. Find this IP address by connecting your robot to the Misty Connect.
- The IP address for a USB-to-Ethernet adapter connected to the USB port on Misty's back. The other end of this adapter must be connected to your router.

Methods

Misty's HTTP API supports GET, POST, and DELETE methods.

- GET requests return data from Misty or download one of Misty's image, audio, or video files to your REST client.
- POST requests upload data to Misty, change system settings, or make Misty do something (for example, drive, move her head, or play a sound).
- DELETE requests uninstall skills, delete one of Misty's image, audio, or video files, or clear system settings.

Requests

POST and DELETE requests in Misty's HTTP API expect a JSON payload. For example:

```
{
  "key0": "value0",
  "key1": "value1",
  "key2": "value2"
}
```

JSON

GET requests do not accept JSON payloads, and query parameters must be included in the URL for the request. To send query parameters with a GET request:

- append a new query parameter to the endpoint with `?`
- assign a value to a query parameter with `=`
- add another query parameter with `&`

For example, a request to the endpoint for the `TakePicture` command might look like this:

```
http://192.138.7.193/api/?base64=true&filename=MyPicture
```

Markup

Unless otherwise specified in this documentation:

- Misty's API expects the `Content-Type` header for POST and DELETE requests to be `application/json`
- the order of key/value pairs in the JSON payload for POST and DELETE requests does not matter
- the order of query parameters in the URL for GET requests does not matter
- for all request types, value strings are case sensitive, and parameter strings are not

Responses

All successful requests return a JSON object with the `status` and `result` of the call.

```
{
  "result": true,
  "status": "Success"
}
```

JSON

A `status` of `Success` indicates Misty received and was able to process the request. A status of `Failed` indicates a problem, and is typically paired with an `error` string (instead of a `result` value).

For most GET requests, the `result` key holds the response data. For example, when you send a request to the `GetImageList` endpoint, the value for `result` is an array of JSON-formatted objects with information about each image saved to Misty's local storage. Alternately, for most POST and DELETE requests, `result` returns a boolean value indicating whether the command was successful.

All requests return a response code with an accompanying status. These include:

Response Code	Status	Meaning
200	`Ok`	The request is valid.
400	`BadArgument`	The request has an invalid payload or query parameter.
404	`NotFound`	A command was not found at this endpoint.
500	`Exception`	Misty threw an exception and failed to handle the request.
503	`ServiceUnavailable`	The operation cannot be performed because Misty's software or firmware is updating.

Additionally, each request returns a `Misty-Command-Version` field in the response header. The value of this field indicates whether the command is considered to be `Alpha`, `Beta`, or `Current`. Commands that are `Current` are considered to be stable. Commands that are `Alpha` and `Beta` are likely to change as Misty's software matures.

Using the API Explorer

From Misty Studio, wou can use the API Explorer to send custom requests to all of the endpoints from Misty's HTTP API in real-time.

The API Explorer generates runnable HTTP-client code samples in JavaScript. These samples use the Fetch API. Additionally, if a REST endpoint has a matching method in Misty's JavaScript API (most do), the API Explorer generates a runnable sample that you can use with Misty's JavaScript SDK.

Example

This section walks through each step in writing the code to send Misty a request from your web browser using the Fetch API. To run this code, place the samples below inside `<script>` tags in an .html file, and open that file in your web browser.

To send an HTTP request with the Fetch API, we call the `fetch()` method. This method takes two arguments:

- `resource` (string): The URL for the request. When using Misty's HTTP API, resources are hosted at `<robot-ip-address>/api/<Endpoint>`.
- `init` (object): An optional argument that defines custom settings to use with the request. In this example, we use this argument to define the HTTP method and any additional parameters to send in the body of the request.

To send a request to Misty, we append the endpoint for the request to the base URL. For example, to get a list of the images stored on Misty, we send a request to the `/images/list` endpoint to call the `GetImageList` command.

For a robot with an IP of `192.168.7.183`, the full resource URL for the `GetImageList` endpoint is as follows:

```
192.168.7.183/api/images/list
```

Markup

When we use the `fetch()` method to send a GET request, we pass in the full command URL as the `resource`. Then, we specify the request method in the object we pass in for the `init` argument. For example, setting up the `fetch()` method to send a request to the `GetImageList` endpoint would look like this:

```
fetch('http://192.168.7.33/api/images/list', {
    method: 'GET'
  })
```

JavaScript

Next, we use a `Promise` to handle the request and response data. We use `Promise.race()` to return a new `Promise` with our call on the `fetch()` method. We then handle the response data by printing it to the console in our web browser.

> **Tip:** The details of using `Promises` are beyond the scope of this documentation. For more information about using `Promises` in your code, see the MDN web documentation.

When we implement the `Promise`, our full `GetImageList` request looks like this. When we load our .html page, the script retrieves Misty's list of images and prints that list to the developer console in our web browser.

```
Promise.race([
    fetch('http://192.168.7.33/api/images/list', {
        method: 'GET'
    }),
    new Promise((_, reject) => setTimeout(() => reject(new
Error('timeout')), 10000))
])
.then(response => response.json())
.then(jsonData => console.log(jsonData))
```

JavaScript

To send a POST request with the Fetch API, we must define a `body` parameter in the `init` object for the `fetch()` method. This `body` is the JSON payload to send along with our request.

For example, to change Misty's LED, we send a POST request to the `/led` endpoint to invoke the `ChangeLED` command. The payload for this request includes key/value pairs for the `red`, `green`, and `blue` RGB values for the new color.

When the following code runs, Misty changes her LED to blue, and the response data prints to the developer console in our web browser:

```
Promise.race([
    fetch('http://192.168.7.33/api/led', {
        method: 'POST',
        body: '{ "red":0,"green":0,"blue":255 }'
    }),
    new Promise((_, reject) => setTimeout(() => reject(new
Error('timeout')), 10000))
])
.then(response => response.json())
.then(jsonData => console.log(jsonData))
```

JavaScript

Getting Live Data from Misty

In addition to getting data via HTTP requests, you can use live data from Misty's sensors and other event types in your remote-running robot application by connecting to Misty's WebSocket server. A WebSocket connection provides a live, continuously updating stream of data from Misty.

> **Tip:** You can directly observe WebSocket data in your web browser's developer console by connecting your robot to Misty Studio. However, to use WebSocket data in a robot application, you'll need to subscribe to it programmatically in your code.

The sections below use the `tofApp.js` code sample from the MistyCommunity GitHub organization to demonstrate how to create a WebSocket connection and subscribe to data from a particular event type. Refer to this section to learn how to use Misty's WebSocket server to register for events in your own remote-running code.

Using Misty's WebSocket Server

To connect to Misty's WebSocket server, use the URL `ws://<robot-ip-address>/pubsub`. Replace `<robot-ip-address>` with the address of the robot from which to stream data. For example:

```
ws://192.168.7.183/pubsub
```

Markup

Formatting the Subscribe Message

When you open a connection to Misty's WebSocket server, you can send a subscription message to receive event messages from a particular event type. You must send this subscription message as a stringified JSON object with the following key/value pairs:

- `Operation` (string) - Use `subscribe` to create a subscription to data from a particular event type. Use `unsubscribe` to end an existing subscription.
- `Type` (string) - The event type that you want to stream data from. See the list of event types in the Event Types documentation.
- `DebounceMs` (int) - How frequently (in milliseconds) to send new event data. Defaults to 250ms. **Note:** Some event types do not provide a constant stream of new data. Instead, these event types send an event message when a particular event occurs (for example, when a bump sensor is pressed.)
- `EventName` (string) - A name of your choosing for the subscription. You can create multiple subscriptions to the same event type, each with their own unique event name.
- `ReturnProperty` (string) - An individual event property to include in the event message. Supports nested properties via dot notation (for example, `"MentalState.Affect.Valence"`). Use `null` to include all event properties in the event message.
- `EventConditions` (array) - Optional. A set of rules about the kind of event messages this subscription can receive. If not included, this subscription receives all messages from the specified event type. Use event conditions to filter out unwanted event messages. Alternatively, use them to specify which sensor you want to receive data from, in cases where you are subscribing to an event type that gets data from more than one sensor. Each object in the `EventConditions` array creates a new conditional statement that each event message must pass in order to be sent to this subscription. Your subscribe message can include as many event conditions as you like. Each object in the `EventConditions` array should have the following key/value pairs:
  - `Property` (string) - The event property to check. The value of this property is compared to the `Value` key below, using the comparison operator passed in for the `Inequality` key.
  - `Inequality` (string) - The comparison operator to use in the conditional check, passed in as a string. Accepts `=>`, `=`, `!=`, `>`, `<`, `>=`, `exists`, `empty`, or `delta`.
  - `Value` (string) - The value to check against.

To demonstrate, let's examine the first block of code from the `tofApp.js` sample. In this sample we create an object called `subscribeMsg` to send to Misty when we open our WebSocket connection. This object creates a new subscription to `TimeOfFlight` event messages. This subscription is named `FrontCenterTimeOfFlight`, and it only receives messages when the value of the `SensorId` property is equal to `toffc` (meaning the message is from the front center time-of-flight sensor).

```
//Create a message to subscribe to the desired WebSocket data.
var subscribeMsg = {
  "Operation": "subscribe", // create a new subscription
  "Type": "TimeOfFlight", // event type to subscribe to
```

```
    "DebounceMs": 100, // send data every 100 milliseconds
    "EventName": "FrontCenterTimeOfFlight", // name of this subscription
    "ReturnProperty": null,
    "EventConditions": [ // Only send messages where SensorId = toffc
      {
        "Property": "SensorId",
        "Inequality": "=",
        "Value": "toffc"
      }
    ]
};

var subMsg = JSON.stringify(subscribeMsg);
```

JavaScript

This subscription starts to get messages immediately, because Misty's time-of-flight sensors are always streaming data. Other event types (for example, `FaceRecognition`) do not start sending data until you start the process that enables that data to be sent (for example, by sending a `StartFaceRecognition` command.)

Formatting the Unsubscribe Message

When you are done using data from a particular subscription, you should send a message to unsubscribe from that event. Leaving unused subscriptions open can lead to performance issues.

To unsubscribe from an event, send a message with an `Operation` value of `unsubscribe`. Pass in the name of the subscription to end as the value for the `EventName` property.

For example, the unsubscribe message for the `FrontCenterTimeOfFlight` event in our `tofApp.js` sample looks like this:

```
//Create a message to unsubscribe to the data when done.
var unsubscribeMsg = {
  "Operation": "unsubscribe",
  "EventName": "FrontCenterTimeOfFlight",
};

var unsubMsg = JSON.stringify(unsubscribeMsg);
```

JavaScript

Opening & Closing a WebSocket

To send a `subscribe` or `unsubscribe` message, we must open a WebSocket connection to Misty. Once this connection is open, we can use it to send data to and read data from Misty's WebSocket server. The remainder of this topic uses examples from the `tofApp.js` sample code to demonstrate the basics of opening and closing a connection to Misty's WebSocket server.

In our `tofApp.js` sample code, we begin by assigning Misty's IP address to a variable:

```
var ip = "<robot-ip-address>"
var socket;
```

JavaScript

We use this IP address to declare the function that manages our WebSocket connection.

```
var ip = "<robot-ip-address>"
var socket;

function startTimeOfFlight() {
  // Create a new WebSocket connection to the robot.
```

```
    socket = new WebSocket("ws://" + ip + "/pubsub");
};
```

JavaScript

When the socket opens, we send the subscribe message from the example above:

```
var ip = "<robot-ip-address>"
var socket;

function startTimeOfFlight() {
    // Create a new WebSocket connection to the robot.
    socket = new WebSocket("ws://" + ip + "/pubsub");

    // When the WebSocket's open, send the subscribe message.
    socket.onopen = function(event) {
      console.log("WebSocket opened.");
      socket.send(subMsg);
    };
};
```

JavaScript

The `tofApp.js` sample code prints ten `TimeOfFlight` event messages, and then closes the WebSocket. To do this, we instantiate a `messageCount` counter variable to increment with each new message. We then use the `socket.onmessage()` function to write the logic that determines what should happen when the app gets data from Misty. When the value of `messageCount` is 10, we send the unsubscribe message from the example above and close the WebSocket connection.

```
var ip = "<robot-ip-address>"
var socket;
var messageCount = 0;

function startTimeOfFlight() {
    // Create a new WebSocket connection to the robot.
    socket = new WebSocket("ws://" + ip + "/pubsub");

    // When the WebSocket's open, send the subscribe message.
    socket.onopen = function(event) {
      console.log("WebSocket opened.");
      socket.send(subMsg);
    };

    // Handle the WebSocket data from the server.
    // Send the unsubscribe message when we're done,
    // then close the socket.
    socket.onmessage = function(event) {
      var message = JSON.parse(event.data).message;
      messageCount += 1;
      console.log(message);
      if (messageCount == 10) {
          socket.send(unsubMsg);
        socket.close();
      }
    };
};
```

JavaScript

> **Tip:** Any given WebSocket connection can be used for multiple data subscriptions. Instead of closing a connection, you can keep it open after sending an unsubscribe message, and only close it when you are done running the application.

Finally, we add some error handling and ask the app to log a message when the connection

is closed:

```
var ip = "<robot-ip-address>"
var socket;
var messageCount = 0;

function startTimeOfFlight() {
    // Create a new WebSocket connection to the robot.
    socket = new WebSocket("ws://" + ip + "/pubsub");

    // When the WebSocket's open, send the subscribe message.
    socket.onopen = function(event) {
      console.log("WebSocket opened.");
      socket.send(subMsg);
    };

    // Handle the WebSocket data from the server.
    // Send the unsubscribe message when we're done,
    // then close the socket.
    socket.onmessage = function(event) {
      var message = JSON.parse(event.data).message;
      messageCount += 1;
      console.log(message);
      if (messageCount == 10) {
        socket.send(unsubMsg);
        socket.close();
      }
    };

    // Handle any errors that occur.
    socket.onerror = function(error) {
      console.log("WebSocket Error: " + error);
    };

    // Do something when the WebSocket is closed.
    socket.onclose = function(event) {
      console.log("WebSocket closed.");
    };
};
```

JavaScript

See the complete `tofApp.js` code sample on GitHub.

Using the LightSocket Helper Tool

The `lightSocket.js` helper tool streamlines opening, connecting, and subscribing to Misty's WebSocket server to receive event messages from the robot in applications that you run from your web browser. Download `lightSocket.js` from GitHub.

We typically recommend placing the `lightSocket.js` file in a "tools" or "assets" folder at the top level of your project directory. It's important to reference the file prior to your application file in your .html markup. For example:

```
<!-- In your HTML file -->
<script src="tools/lightSocket.js"></script>
<script src="app.js"></script>
```

HTML

The first step in using `lightSocket.js` is to create an instance of the `LightSocket` class, passing in your robot's IP address. Then, call the `socket.Connect()` method to open a WebSocket connection.

```
let socket = new LightSocket(ip);
socket.Connect();
```

JavaScript

To subscribe to an event via LightSocket, we call the `Subscribe()` method from our new `socket` class. The arguments passed to the function correspond to the properties of the subscribe message described in Formatting the Subscribe Message. For example:

```
// Syntax
socket.Subscribe = function (eventName, msgType, debounceMs, property,
inequality, value, returnProperty, eventCallback)
```

JavaScript

Here, we subscribe to `TimeOfFlight` event messages for the center time-of-flight sensor, and log the data as we receive it:

```
socket.Subscribe("CenterTimeOfFlight", "TimeOfFlight", 100,
"SensorPosition", "=", "Center", null, function(data) {
    console.log(data);
});
```

JavaScript

To prevent performance issues, we recommend ending a subscription when you are done using its data. At the end of your script, be sure to call the `Unsubscribe()` method:

```
socket.Unsubscribe("CenterTimeOfFlight");
```

JavaScript